

Voice-Over Script — RBAC-Enforced Financial Data Agent

AZENTIO · AI AGENT DEVELOPER · TARGET 7–8 MINUTES · MADHAV SHARMA

How to use. Grey blocks are what you **say** — read them close to verbatim, they are timed. Dashed blocks are what you **do** on screen. Speak at a normal pace, roughly 150 words a minute; do not rush the demo section. If you overrun, cut section 4.

BEFORE RECORDING

Run `python scripts/serve.py`. Open the console. Have these three links ready in tabs so nothing is typed live:

`/?role=CEO&q=What was net sales in FY2025?`

`/?role=CTO&q=What is revenue per employee for FY2025?`

`/?role=ANALYST&q=What was net sales in FY2023?`

Also open a terminal at the project root.

1 · The problem (0:00 – 0:50)

SCREEN

Console open, nothing asked yet.

SAY

Hello. I'm Madhav Sharma, and this is my submission for the AI Agent Developer role.

I built a financial-data agent over Apple's public SEC filings. You ask a question in plain English, and it answers — but the answer depends on **who is asking**.

Most of that is a retrieval problem, and it's well understood. The part that isn't is this: **you can leak a restricted number without ever printing it**. If I tell you revenue, and I tell you revenue per employee, I've told you headcount. Hiding a value while publishing a function of it is not hiding it.

So I built the access control first, and the answering around it.

2 · Architecture (0:50 – 2:10)

SCREEN

Show the console layout, or the diagram in `docs/ARCHITECTURE.md`.

SAY

The organising principle is **deterministic core, LLM at the edge**. Every figure, every access decision, every refusal is produced by ordinary Python. The model only interprets phrasing and words the answer.

Three things follow from that. It's testable, because an access decision is a pure function of a role and a plan. It's reproducible — the same question gives the same number every time. And it runs with **no API key at all**, which matters because half the grade is "does it work", and that shouldn't depend on someone's billing page.

The pipeline is five stages. Ingestion turns PDFs and spreadsheets into two typed records — a Fact, which is a number with its source and page, and a Chunk, which is a passage. Both carry a sensitivity tag assigned deterministically from a config file, never by a model.

Then everything — everything — reads through one gate.

3 · The demonstration (2:10 – 3:50) ← the important part

SCREEN

Open the **CEO** link. Let the pipeline animate. Pause on the figure.

SAY

As CEO: plan, guard, retrieve, compose, audit — all five green. Net sales for FY2025, four hundred sixteen thousand one hundred sixty-one million dollars, cited to the filing and the page it came from.

SCREEN

Open the **CTO** link. Let it stop at GUARD. **Pause two seconds here.**

SAY

Now the same system, as CTO, asking for revenue per employee. The plan is built — and the guard rejects it. Retrieve and compose grey out: **"not reached — nothing was fetched."**

That's the part I want to be clear about. The CTO isn't being shown an answer with the headcount removed. **The query never ran.** The restricted operand was never read.

That's what "enforce it at the data layer" means. Telling an agent not to read something is a request. Not giving it a tool that can is a guarantee.

SCREEN

Open the **ANALYST** link.

SAY

And a third role, refused for a completely different reason — the year is outside its window. ANALYST is restricted on two dimensions, subject **and** time, so this is a real permission model rather than three if-statements.

4 · Why the interface looks like this (3:50 – 4:30)

SAY

I deliberately didn't build a chat box. In a system whose whole point is access control, the **decision** is the interesting object, not the answer.

A chat transcript would render that refusal as just another message — indistinguishable from the model choosing not to answer. That's exactly the confusion this system exists to remove, so I didn't want the interface reintroducing it.

The audit trail underneath records every decision, allows as well as denials — because a log of only refusals can't answer "what did this person actually see", which is the first question an auditor asks. And it records decisions only, never document text, so the log doesn't become a second unguarded copy of the thing the gate protects.

5 · Feedback and the bonus (4:30 – 5:30)

SCREEN

Terminal: `python scripts/demo.py`. Scroll to **ACT 4**.

SAY

For the feedback loop — same question asked twice, either side of a down-vote and a correction. Ranking before, ranking after, printed. Storing feedback is not learning from it; **this diff is the difference**.

The security property matters more than the mechanism: feedback runs strictly **after** the gate. You cannot up-vote your way into restricted data. Thirty up-votes on a compensation passage don't unlock it — and in fact they're never consulted, because the guard refused the plan before retrieval ran.

SCREEN

Scroll to **ACT 5**.

SAY

The bonus was prompt injection. I put a deliberately poisoned document in the corpus — it reads like a supplier report and carries hidden instructions.

It's quarantined at ingest, and the gate drops it for **every** role including CEO, because a document carrying an attack is unsafe for everyone. Quarantine is a safety rule, not a permission.

I also handle injection in user input — the clause is cut out, the real question is still answered, and the attempt is logged.

And I'd rather be honest about this: four layers, and only two are strong. Quarantine is structural, the no-text-to-execution path is absolute, but the prompt-level defences are mitigations. Anyone claiming prompt injection is solved is overselling it.

6 · What I'd do differently (5:30 – 6:20)

SAY

Three things.

Parse the XBRL instance document instead of its HTML rendering. About twelve per cent of my deep footnote metrics are still ambiguous, because several sub-tables repeat identical row labels like "Total". XBRL element names are globally unique by construction, so that whole class of problem disappears. No headline metric is affected, but it's the first thing I'd fix.

Real authentication. I assert identity rather than authenticating it. That was deliberate — the brief asks for enforcement at the data layer, so I spent the time making that layer unbypassable instead of building a login screen. Swapping it in changes one function.

A held-out evaluation set. I can prove access control with tests. I can't put a number on answer quality, and I'd want one before anyone relied on this.

7 · What breaks at 100× (6:20 – 7:15)

SAY

The in-memory index and SQLite's single writer go first. Both become a server-backed store — and the important part of that migration is that the permission predicate has to push **down** into whatever replaces them. If it degrades into a post-filter, the side channel comes back: result counts and latency leak what the rows don't.

The whole-corpus rebuild is three seconds now and unusable at a hundred times — it needs to be incremental and content-hash keyed. PDF parsing is the throughput ceiling at seconds per page, so it becomes a queue with workers. Feedback re-ranking is a linear scan that needs its own index.

But the one I'd actually worry about is the **tag taxonomy**. It's hand-maintained. At a hundred times the documents its coverage rots quietly — and untagged data defaults into **visible**. That's the failure that doesn't announce itself, and it needs an owner and an alert on untagged ingestion, not just more code.

8 · Close (7:15 – 7:50)

SAY

To summarise the shape of it: ingest both formats into tagged, typed records; precompute anything whose failure would be silent; put one gate in front of everything and filter **before** retrieval, not after; refuse plans whose answer would leak by arithmetic; and keep the model at the edge so the system stays testable and runs without a key.

Two hundred and twenty-one tests, and a decisions file recording eighty-nine choices with the alternative I rejected for each — because the reasoning is the part worth reviewing.

Everything I've said is checkable in the repository, and I'd like you to check it. Thank you.

Delivery notes

DO	WHY
Spend the first minute on the leak problem	It frames everything after it and separates this from a RAG demo
Say " the query never ran "	The single most important sentence in the recording
Pause 2 seconds on the stopped pipeline	Let the viewer see it halt
Volunteer the limitations in section 6	Raising them yourself is worth more than surviving them
Don't narrate code line by line	They can read it. Explain choices.
If overrunning, cut section 4	Least load-bearing minute

Numbers you may need. 221 tests · 28 commits · 89 decisions · 4,685 facts · 737 chunks · 467 PDF pages · 8 filings · 3 roles · 7 tags · ~3 s rebuild · 51 ms mean answer.

Net sales FY2025 \$416,161M · FY2024 \$391,035M · FY2023 \$383,285M. Headcount 166,000 / 164,000 / 161,000.